IS 1210
Database Systems

Lecturer 3 – Normalization (Part 3)

MVPT Lakshika,

Lecturer (Probationary),

Department of Information Systems Engineering, UCSC.

Contact: tlv@ucsc.cmb.ac.lk

Outline of this lecture

Process of Normalization

1. Unnormalized Form (UNF)
2. First Normal Form (1NF)
3. Second Normal Form (2NF)
4. Third Normal Form (3NF)
5. Boyce-Codd Normal Form (BCNF)

Process of Normalization

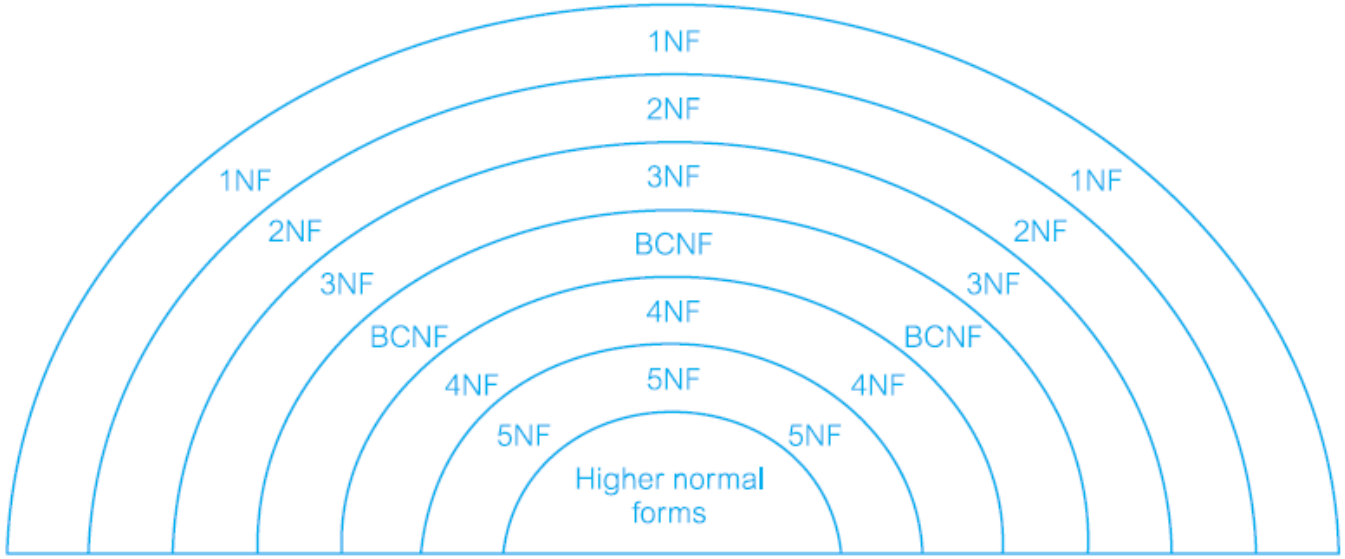
Introduction

- Normalization is a formal technique for analyzing relations based on their primary key (or candidate keys) and functional dependencies.
- The technique involves a series of rules that can be used to test individual relations so that a database can be normalized to any degree.
- When a requirement is not met, the relation violating the requirement must be decomposed into relations that individually meet the requirements of normalization.
- As normalization proceeds, the relations become progressively more restricted (stronger) in format and also less vulnerable to update anomalies.
- But, performance degrades.

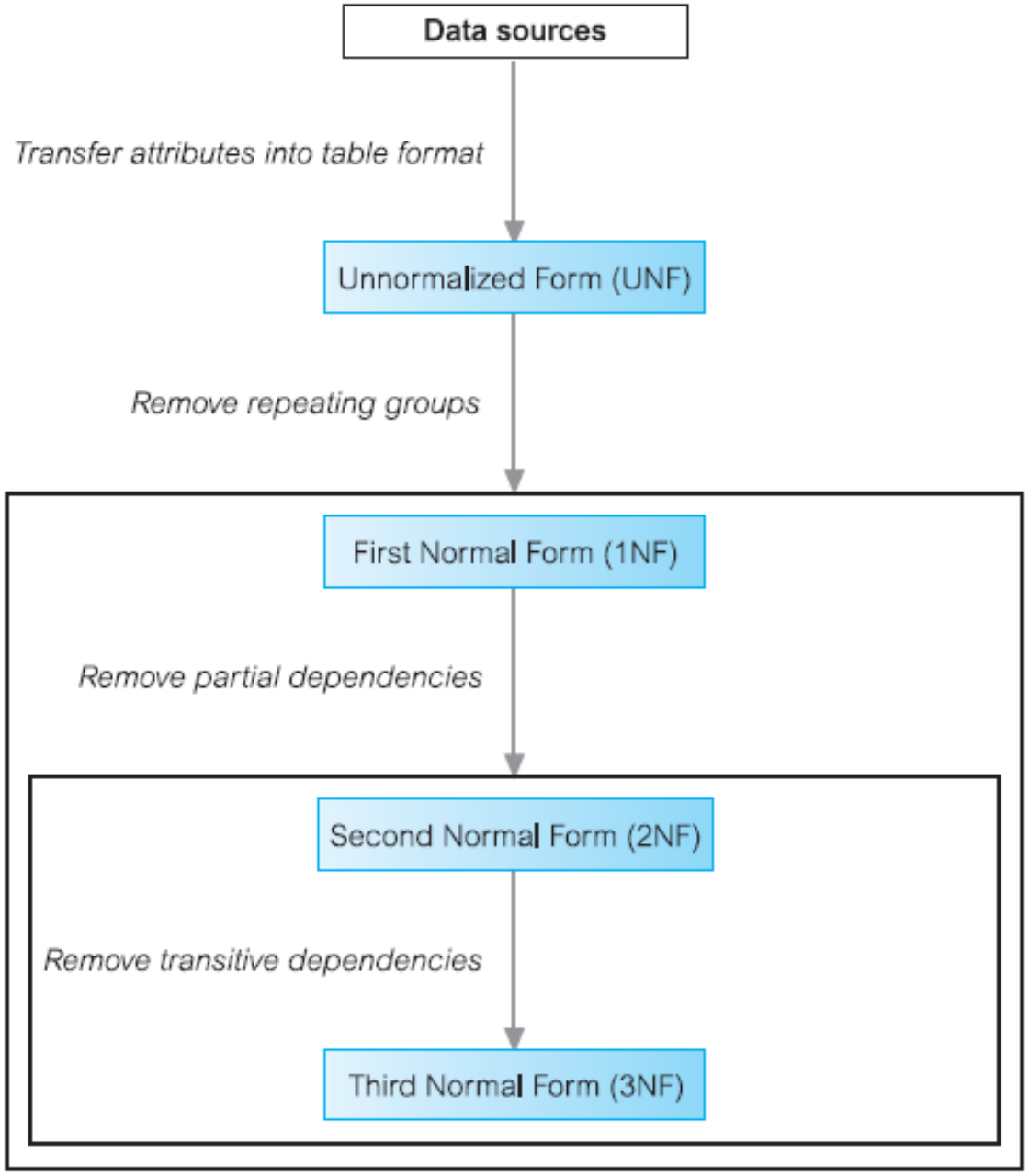
Relationship between the normal forms

- **Three main normal forms**: First Normal Form (1NF), Second Normal Form (2NF), and Third Normal Form (3NF).
- Subsequently, R. Boyce and E.F. Codd introduced a stronger definition of 3NF called Boyce–Codd Normal Form (BCNF).
- Higher normal forms that go beyond BCNF were introduced later, such as 4NF and 5NF.

- However, these later normal forms deal with very rare situations.
- Figure shows that some 1NF relations are also in 2NF and that some 2NF relations are also in 3NF, and so on.



Process of Normalization



1. Unnormalized Form (UNF)

Unnormalized Form (UNF)

- The process of normalization starts with first transferring the data from the sources into a table format with rows and columns.
- In this format, the table is in Unnormalized Form and is referred to as an unnormalized table.
  - Contains one or more repeating groups in the tables.
  - A repeating group is an attribute, or group of attributes, within a table that occurs with multiple values for a single occurrence of the key attribute(s) for that table.

Removing repeating groups

There are two common approaches to removing repeating groups from unnormalized tables.

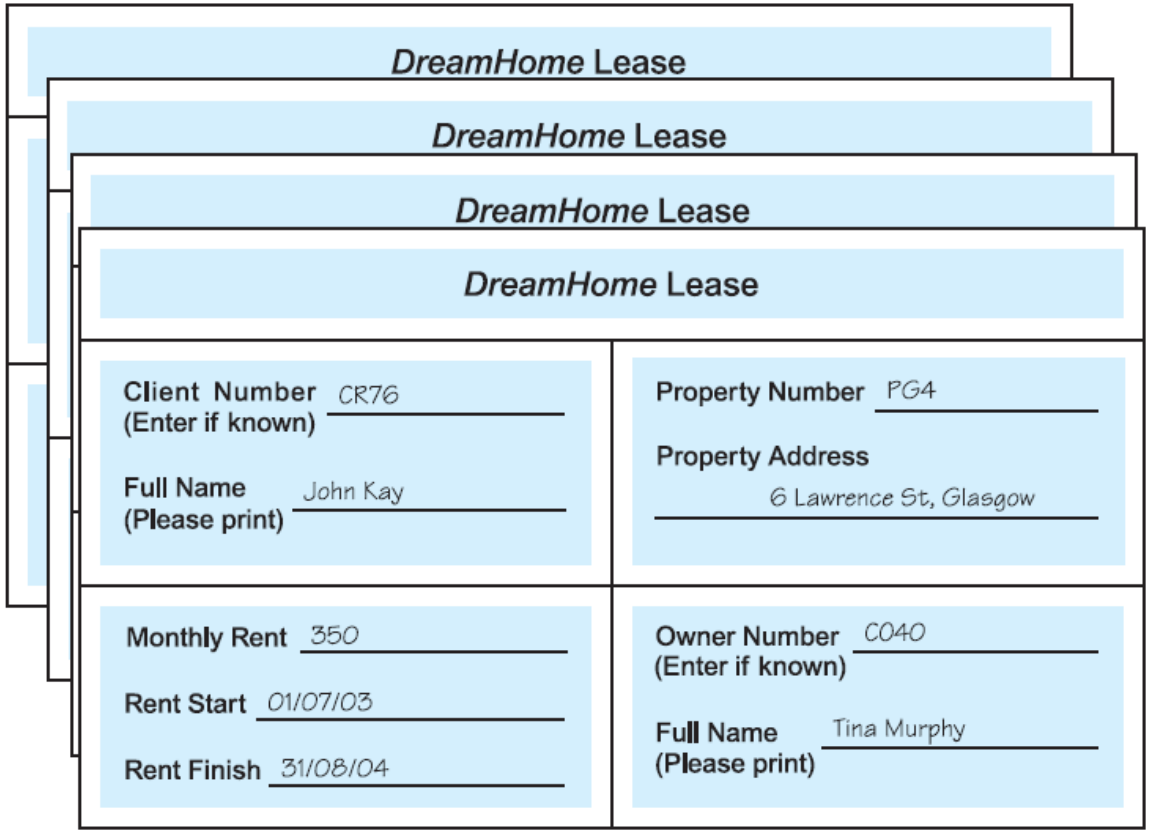
1. Flattening the table - By entering appropriate data in the empty columns of rows containing the repeating data (fill in the blanks by duplicating the nonrepeating data, where required)
2. By placing the repeating data, along with a copy of the original key attribute(s), in a separate relation.

   - Sometimes the unnormalized table may contain more than one repeating group, or repeating groups within repeating groups.

   - In such cases, this approach is applied repeatedly until no repeating groups remain.

Example - UNF

A collection of (simplified) *DreamHome* leases →



<u>Assumption:</u> A client rents a given property only once and cannot rent more than one property at any one time.

ClientRental

clientNo	cName	propertyNo	pAddress	rentStart	rentFinish	rent	ownerNo	oName
CR76	John Kay	PG4	6 Lawrence St, Glasgow	1-Jul-03	31-Aug-04	350	CO40	Tina Murphy
		PG16	5 Novar Dr, Glasgow	1-Sep-04	1-Sep-05	450	CO93	Tony Shaw
CR56	Aline Stewart	PG4	6 Lawrence St, Glasgow	1-Sep-02	10-June-03	350	CO40	Tina Murphy
		PG36	2 Manor Rd, Glasgow	10-Oct-03	1-Dec-04	375	CO93	Tony Shaw
		PG16	5 Novar Dr, Glasgow	1-Nov-05	10-Aug-06	450	CO93	Tony Shaw

← *ClientRental* unnormalized table

The key attribute for the *ClientRental* unnormalized table is clientNo.

2. First Normal Form (1NF)

First Normal Form (1NF)

1NF: A relation in which the intersection of each row and column contains **one and only one value.**

- The structure of the repeating groups from the previous example:

Repeating Group = (propertyNo, pAddress, rentStart, rentFinish, rent, ownerNo, oName)

- UNF to 1NF: ensure that there is a single value at the intersection of each row and column.

Approach 1: Flattening the table

- Remove the repeating group by entering the appropriate client data into each row.

Approach 2:

- Remove the repeating group by placing the repeating data, along with a copy of the original key attribute, in a separate relation.

Example – UNF to 1NF

Approach 1: Flattening the table

ClientRental unnormalized table →

ClientRental

Repeating Group

clientNo	cName	propertyNo	pAddress	rentStart	rentFinish	rent	ownerNo	oName
CR76	John Kay	PG4	6 Lawrence St, Glasgow	1-Jul-03	31-Aug-04	350	CO40	Tina Murphy
		PG16	5 Novar Dr, Glasgow	1-Sep-04	1-Sep-05	450	CO93	Tony Shaw
CR56	Aline Stewart	PG4	6 Lawrence St, Glasgow	1-Sep-02	10-June-03	350	CO40	Tina Murphy
		PG36	2 Manor Rd, Glasgow	10-Oct-03	1-Dec-04	375	CO93	Tony Shaw
		PG16	5 Novar Dr, Glasgow	1-Nov-05	10-Aug-06	450	CO93	Tony Shaw

ClientRental

clientNo	propertyNo	cName	pAddress	rentStart	rentFinish	rent	ownerNo	oName
CR76	PG4	John Kay	6 Lawrence St, Glasgow	1-Jul-03	31-Aug-04	350	CO40	Tina Murphy
CR76	PG16	John Kay	5 Novar Dr, Glasgow	1-Sep-04	1-Sep-05	450	CO93	Tony Shaw
CR56	PG4	Aline Stewart	6 Lawrence St, Glasgow	1-Sep-02	10-Jun-03	350	CO40	Tina Murphy
CR56	PG36	Aline Stewart	2 Manor Rd, Glasgow	10-Oct-03	1-Dec-04	375	CO93	Tony Shaw
CR56	PG16	Aline Stewart	5 Novar Dr, Glasgow	1-Nov-05	10-Aug-06	450	CO93	Tony Shaw

← The resulting first normal form (INF) *ClientRental* relation

Example – UNF to 1NF

Approach 2: Introduce alternative 1NF relations.

ClientRental unnormalized table →

ClientRental

Repeating Group

clientNo	cName	propertyNo	pAddress	rentStart	rentFinish	rent	ownerNo	oName
CR76	John Kay	PG4	6 Lawrence St, Glasgow	1-Jul-03	31-Aug-04	350	CO40	Tina Murphy
		PG16	5 Novar Dr, Glasgow	1-Sep-04	1-Sep-05	450	CO93	Tony Shaw
CR56	Aline Stewart	PG4	6 Lawrence St, Glasgow	1-Sep-02	10-June-03	350	CO40	Tina Murphy
		PG36	2 Manor Rd, Glasgow	10-Oct-03	1-Dec-04	375	CO93	Tony Shaw
		PG16	5 Novar Dr, Glasgow	1-Nov-05	10-Aug-06	450	CO93	Tony Shaw

Client

clientNo	cName
CR76	John Kay
CR56	Aline Stewart

PropertyRentalOwner

clientNo	propertyNo	pAddress	rentStart	rentFinish	rent	ownerNo	oName
CR76	PG4	6 Lawrence St, Glasgow	1-Jul-03	31-Aug-04	350	CO40	Tina Murphy
CR76	PG16	5 Novar Dr, Glasgow	1-Sep-04	1-Sep-05	450	CO93	Tony Shaw
CR56	PG4	6 Lawrence St, Glasgow	1-Sep-02	10-Jun-03	350	CO40	Tina Murphy
CR56	PG36	2 Manor Rd, Glasgow	10-Oct-03	1-Dec-04	375	CO93	Tony Shaw
CR56	PG16	5 Novar Dr, Glasgow	1-Nov-05	10-Aug-06	450	CO93	Tony Shaw

← The resulting *Client* and *PropertyRentalOwner* relations are both in 1NF as there is a single value at the intersection of each row and column.

1NF

- The resulting tables from both previous approaches are now referred to as 1NF relations containing atomic (or single) values at the intersection of each row and column.
- Approach 1 introduces more redundancy into the original UNF table, whereas approach 2 creates two or more relations with less redundancy than in the original UNF table.

  Thus, approach 2 moves the original UNF table further along the normalization process than approach 1.
- No matter which initial approach is taken, the original UNF table will be normalized into the same set of relations at the end.

3. Second Normal Form (2NF)

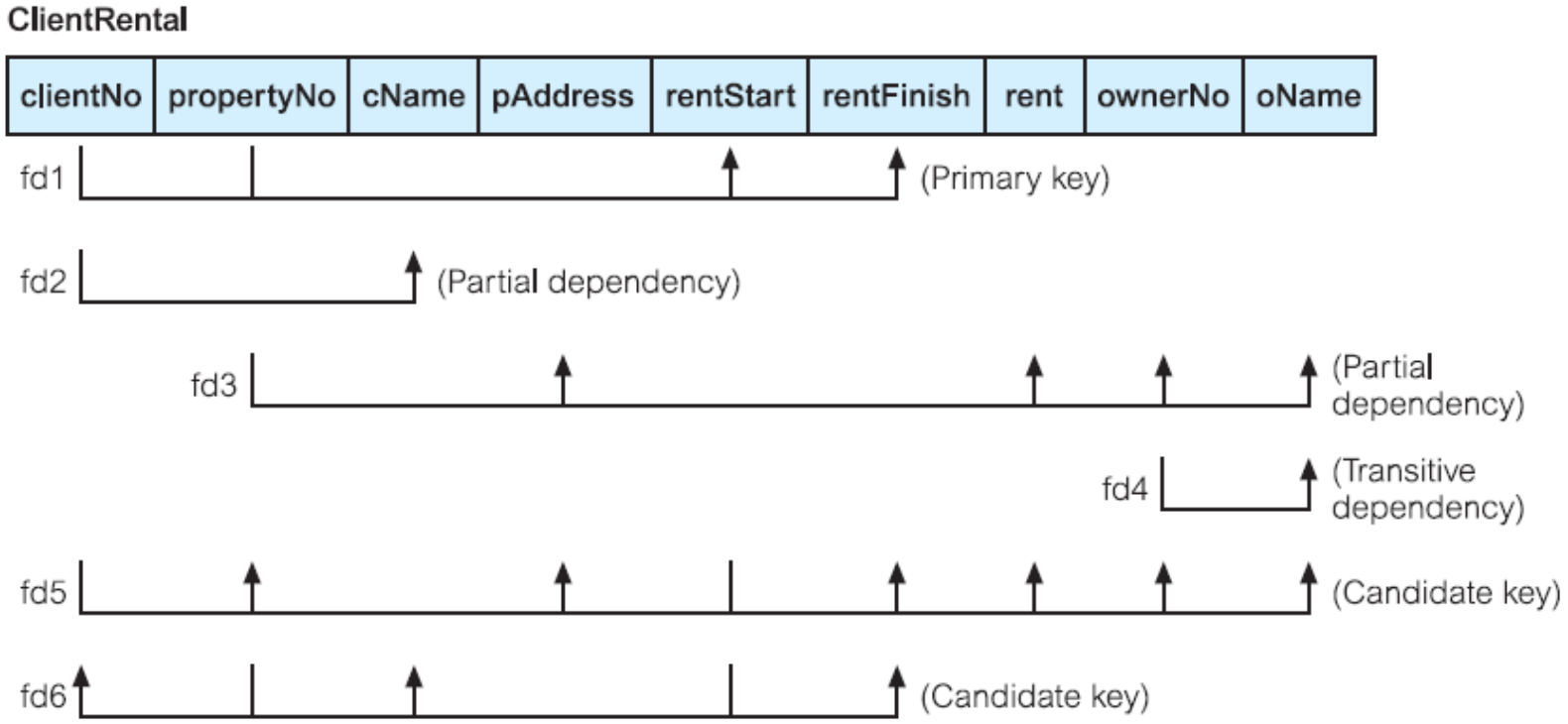
Second Normal Form (2NF)

2NF: A relation that is in 1NF and every non-primary-key attribute is **fully functionally dependent** on the primary key.

- 2NF is based on the concept of full functional dependency.
- 2NF applies to relations with composite keys (relations with a primary key composed of two or more attributes).
- A relation with a single-attribute primary key is automatically in at least 2NF.
  - A relation that is not in 2NF may suffer from the update anomalies.
- The normalization of 1NF relations to 2NF involves the removal of partial dependencies.
  - If a partial dependency exists, remove the partially dependent attribute(s) from the relation by placing them in a new relation along with a copy of their determinant.

Example

Functional dependencies (fd1 to fd6) for the *ClientRental* relation →



fd1	clientNo, propertyNo → rentStart, rentFinish	(Primary key)
fd2	clientNo → cName	(Partial dependency)
fd3	propertyNo → pAddress, rent, ownerNo, oName	(Partial dependency)
fd4	ownerNo → oName	(Transitive dependency)
fd5	clientNo, rentStart → propertyNo, pAddress, rentFinish, rent, ownerNo, oName	(Candidate key)
fd6	propertyNo, rentStart → clientNo, cName, rentFinish	(Candidate key)

In this example, we assume that the *rentFinish* attribute is not appropriate as a component of a candidate key, as it may contain nulls.

Example

Using approach 1

- Use the functional dependencies to identify candidate keys for the *ClientRental* relation.

  Composite keys: (clientNo, propertyNo), (clientNo, rentStart), and (propertyNo, rentStart).

- Next, select the primary key for the relation: (clientNo, propertyNo)

  - For clarity, place the attributes that make up the primary key together at the LHS of the relation.

  - Now, the ClientRental relation is defined as follows:

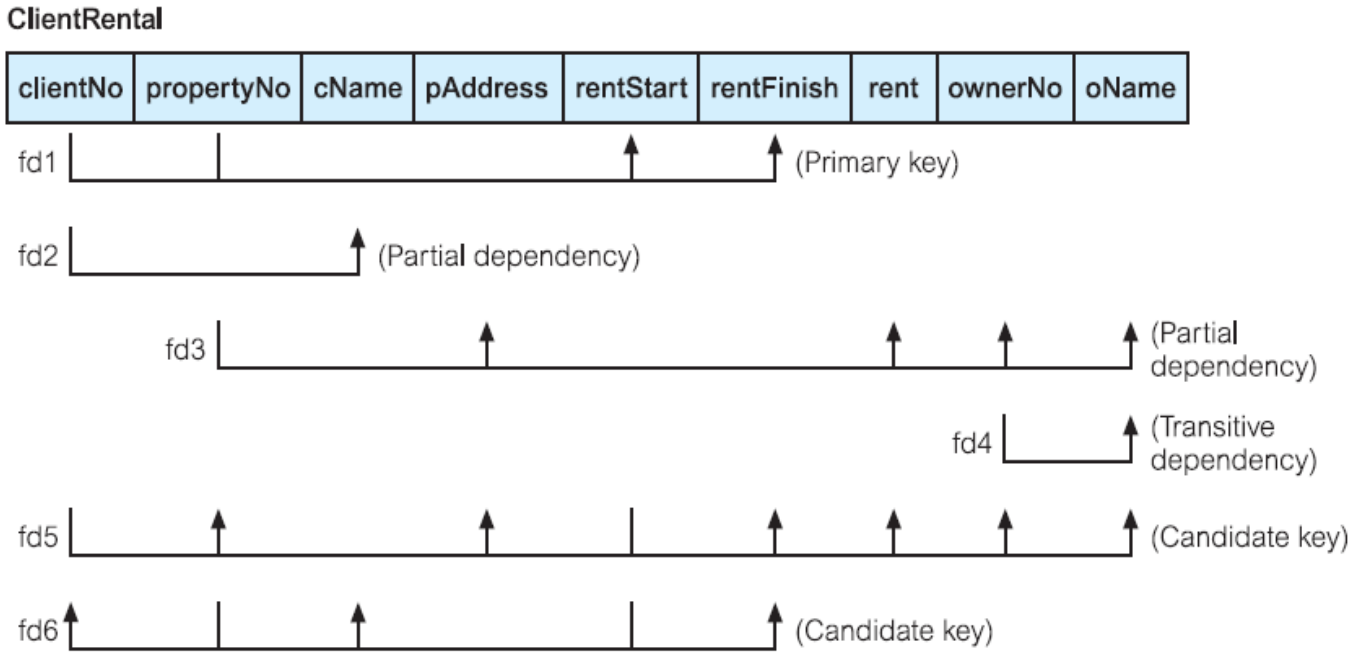
    ClientRental (<u>clientNo,</u> <u>propertyNo</u>, cName, pAddress, rentStart, rentFinish, rent, ownerNo, oName)

- The 1NF relation would be subject to the update anomalies. To remove some of these, you must transform the relation into 2NF.

1NF to 2NF

- 1NF to 2NF involves the removal of partial dependencies.
- Remove the partially dependent attribute(s) from the relation by placing them in a new relation along with a copy of their determinant.

Primary Key : (clientNo, propertyNo)



fd1	clientNo, propertyNo → rentStart, rentFinish	(Primary key)
fd2	clientNo → cName	(Partial dependency)
fd3	propertyNo → pAddress, rent, ownerNo, oName	(Partial dependency)
fd4	ownerNo → oName	(Transitive dependency)
fd5	clientNo, rentStart → propertyNo, pAddress, rentFinish, rent, ownerNo, oName	(Candidate key)
fd6	propertyNo, rentStart → clientNo, cName, rentFinish	(Candidate key)

1NF to 2NF

Primary Key : (clientNo, propertyNo)

- The attribute cName is partially dependent on the primary key (depends on only the clientNo attribute [see fd2]).
- The property attributes (pAddress, rent, ownerNo, oName) are partially dependent on the primary key (depends on only the propertyNo [see fd3]).
- The property rented attributes (rentStart and rentFinish) are fully dependent on the whole primary key (see fd1).
- The identification of partial dependencies within the *ClientRental* relation indicates that the relation is not in 2NF.
- To transform the *ClientRental* relation into 2NF requires the creation of new relations so that the non-primary-key attributes are removed, along with a copy of the part of the primary key on which they are fully functionally dependent.

1NF to 2NF

ClientRental

clientNo	propertyNo	cName	pAddress	rentStart	rentFinish	rent	ownerNo	oName
CR76	PG4	John Kay	6 Lawrence St, Glasgow	1-Jul-03	31-Aug-04	350	CO40	Tina Murphy
CR76	PG16	John Kay	5 Novar Dr, Glasgow	1-Sep-04	1-Sep-05	450	CO93	Tony Shaw
CR56	PG4	Aline Stewart	6 Lawrence St, Glasgow	1-Sep-02	10-Jun-03	350	CO40	Tina Murphy
CR56	PG36	Aline Stewart	2 Manor Rd, Glasgow	10-Oct-03	1-Dec-04	375	CO93	Tony Shaw
CR56	PG16	Aline Stewart	5 Novar Dr, Glasgow	1-Nov-05	10-Aug-06	450	CO93	Tony Shaw

Client

clientNo	cName
CR76	John Kay
CR56	Aline Stewart

Rental

clientNo	propertyNo	rentStart	rentFinish
CR76	PG4	1-Jul-03	31-Aug-04
CR76	PG16	1-Sep-04	1-Sep-05
CR56	PG4	1-Sep-02	10-Jun-03
CR56	PG36	10-Oct-03	1-Dec-04
CR56	PG16	1-Nov-05	10-Aug-06

PropertyOwner

propertyNo	pAddress	rent	ownerNo	oName
PG4	6 Lawrence St, Glasgow	350	CO40	Tina Murphy
PG16	5 Novar Dr, Glasgow	450	CO93	Tony Shaw
PG36	2 Manor Rd, Glasgow	375	CO93	Tony Shaw

- The result is the creation of three new relations called Client, Rental, and PropertyOwner.
- These three relations are in 2NF as every non primary-key attribute is fully functionally dependent on the primary key of the relation.

The relations have the following form:

Client (clientNo, cName)

Rental (clientNo, propertyNo, rentStart, rentFinish)

PropertyOwner (propertyNo, pAddress, rent, ownerNo, oName)

Example

Using the approach 2:

- Using the functional dependencies identified earlier, you can identify a primary key for the *Client* and *PropertyRentalOwner* relations.
- The format of the resulting 1NF relations are as follows:

Client (clientNo, cName)

PropertyRentalOwner (clientNo, propertyNo, pAddress, rentStart, rentFinish, rent, ownerNo, oName)

- The property attributes (pAddress, rent, ownerNo, oName) are partially dependent on the primary key (depends on only the propertyNo [see fd3]).
- The property rented attributes (rentStart and rentFinish) are fully dependent on the whole primary key (see fd1).

1NF to 2NF

Using the approach 2:

- The identification of partial dependencies within the PropertyRentalOwner relation indicates that the relation is not in 2NF.
- To transform the PropertyRentalOwner relation into 2NF requires the creation of new relations so that the non-primary-key attributes are removed along with a copy of the part of the primary key on which they are fully functionally dependent.
- The result is the creation of three new relations called Client, Rental, and PropertyOwner.
- These three relations are in 2NF as every non primary-key attribute is fully functionally dependent on the primary key of the relation.
- The relations have the following form:

Client (clientNo, cName)

Rental (clientNo, propertyNo, rentStart, rentFinish)

PropertyOwner (propertyNo, pAddress, rent, ownerNo, oName)

4. Third Normal Form (3NF)

Third Normal Form (3NF)

3NF: A relation that is in 1NF and 2NF and in which no non-primary-key attribute is **transitively dependent** on the primary key.

- Although 2NF relations have less redundancy than those in 1NF, they may still suffer from update anomalies.

Example: To update the name of an owner, Tony Shaw (ownerNo CO93), you have to update two tuples in the *PropertyOwner* relation.
- If you update only one tuple and not the other, the database would be in an inconsistent state.
- This update anomaly is caused by a transitive dependency.
- You need to remove such dependencies by progressing to 3NF.

PropertyOwner

propertyNo	pAddress	rent	ownerNo	oName
PG4	6 Lawrence St, Glasgow	350	CO40	Tina Murphy
PG16	5 Novar Dr, Glasgow	450	CO93	Tony Shaw
PG36	2 Manor Rd, Glasgow	375	CO93	Tony Shaw

2NF to 3NF

- 2NF to 3NF involves the removal of transitive dependencies.
- If a transitive dependency exists, remove the transitively dependent attribute(s) from the relation by placing the attributes(s) in a new relation along with a copy of the determinant.

These dependencies have altered compared with the original functional dependencies.

fd1	clientNo, propertyNo → rentStart, rentFinish	(Primary key)
fd2	clientNo → cName	(Partial dependency)
fd3	propertyNo → pAddress, rent, ownerNo, oName	(Partial dependency)
fd4	ownerNo → oName	(Transitive dependency)
fd5	clientNo, rentStart → propertyNo, pAddress, rentFinish, rent, ownerNo, oName	(Candidate key)
fd6	propertyNo, rentStart → clientNo, cName, rentFinish	(Candidate key)

<u>Client</u>

fd2	clientNo → cName	(Primary key)

<u>Rental</u>

fd1	clientNo, propertyNo → rentStart, rentFinish	(Primary key)
fd5′	clientNo, rentStart → propertyNo, rentFinish	(Candidate key)
fd6′	propertyNo, rentStart → clientNo, rentFinish	(Candidate key)

<u>PropertyOwner</u>

fd3	propertyNo → pAddress, rent, ownerNo, oName	(Primary key)
fd4	ownerNo → oName	(Transitive dependency)

2NF to 3NF

Resulted relations after 2NF →

Client (clientNo, cName)
Rental (clientNo, propertyNo, rentStart, rentFinish)
PropertyOwner (propertyNo, pAddress, rent, ownerNo, oName)

- All the non-primary-key attributes within the Client and Rental relations are functionally dependent on only their primary keys.
  - Those relations have no transitive dependencies and are therefore already in 3NF.
- All the non-primary-key attributes within the PropertyOwner relation are functionally dependent on the primary key, except oName, which is transitively dependent on ownerNo (see fd4).
  - To transform the PropertyOwner relation into 3NF, you must remove this transitive dependency by creating two new relations called PropertyForRent and Owner.

2NF to 3NF

- The new relations have the form:

  PropertyForRent (propertyNo, pAddress, rent, ownerNo)

  Owner (ownerNo, oName)

- The PropertyForRent and Owner relations are in 3NF as there are no further transitive dependencies on the primary key.

Client

clientNo	cName
CR76	John Kay
CR56	Aline Stewart

Rental

clientNo	propertyNo	rentStart	rentFinish
CR76	PG4	1-Jul-03	31-Aug-04
CR76	PG16	1-Sep-04	1-Sep-05
CR56	PG4	1-Sep-02	10-Jun-03
CR56	PG36	10-Oct-03	1-Dec-04
CR56	PG16	1-Nov-05	10-Aug-06

PropertyOwner

propertyNo	pAddress	rent	ownerNo	oName
PG4	6 Lawrence St, Glasgow	350	CO40	Tina Murphy
PG16	5 Novar Dr, Glasgow	450	CO93	Tony Shaw
PG36	2 Manor Rd, Glasgow	375	CO93	Tony Shaw

Client

clientNo	cName
CR76	John Kay
CR56	Aline Stewart

Rental

clientNo	propertyNo	rentStart	rentFinish
CR76	PG4	1-Jul-03	31-Aug-04
CR76	PG16	1-Sep-04	1-Sep-05
CR56	PG4	1-Sep-02	10-Jun-03
CR56	PG36	10-Oct-03	1-Dec-04
CR56	PG16	1-Nov-05	10-Aug-06

PropertyForRent

propertyNo	pAddress	rent	ownerNo
PG4	6 Lawrence St, Glasgow	350	CO40
PG16	5 Novar Dr, Glasgow	450	CO93
PG36	2 Manor Rd, Glasgow	375	CO93

Owner

ownerNo	oName
CO40	Tina Murphy
CO93	Tony Shaw

Summary of the 3NF relations

- The ClientRental relation has been transformed by the process of normalization into four relations in 3NF.

ClientRental (clientNo, propertyNo, cName, pAddress, rentStart, rentFinish, rent, ownerNo, oName)

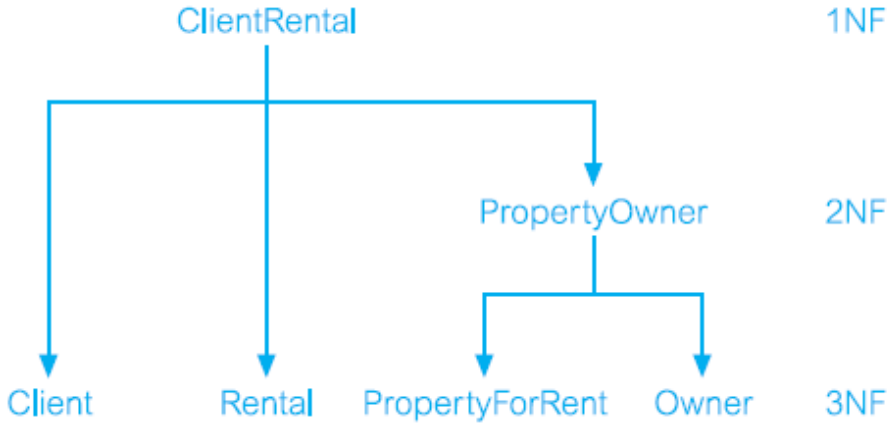
- The resulting 3NF relations have the form:

Client (clientNo, cName)

Rental (clientNo, propertyNo, rentStart, rentFinish)

PropertyForRent (propertyNo, pAddress, rent, ownerNo)

Owner (ownerNo, oName)



- The original ClientRental relation can be recreated by joining the Client, Rental, PropertyForRent, and Owner relations through the primary key/foreign key mechanism.

Summary of the 3NF relations

ClientRental

clientNo	cName	propertyNo	pAddress	rentStart	rentFinish	rent	ownerNo	oName
CR76	John Kay	PG4	6 Lawrence St, Glasgow	1-Jul-03	31-Aug-04	350	CO40	Tina Murphy
		PG16	5 Novar Dr, Glasgow	1-Sep-04	1-Sep-05	450	CO93	Tony Shaw
CR56	Aline Stewart	PG4	6 Lawrence St, Glasgow	1-Sep-02	10-June-03	350	CO40	Tina Murphy
		PG36	2 Manor Rd, Glasgow	10-Oct-03	1-Dec-04	375	CO93	Tony Shaw
		PG16	5 Novar Dr, Glasgow	1-Nov-05	10-Aug-06	450	CO93	Tony Shaw

Client

clientNo	cName
CR76	John Kay
CR56	Aline Stewart

Rental

clientNo	propertyNo	rentStart	rentFinish
CR76	PG4	1-Jul-03	31-Aug-04
CR76	PG16	1-Sep-04	1-Sep-05
CR56	PG4	1-Sep-02	10-Jun-03
CR56	PG36	10-Oct-03	1-Dec-04
CR56	PG16	1-Nov-05	10-Aug-06

PropertyForRent

propertyNo	pAddress	rent	ownerNo
PG4	6 Lawrence St, Glasgow	350	CO40
PG16	5 Novar Dr, Glasgow	450	CO93
PG36	2 Manor Rd, Glasgow	375	CO93

Owner

ownerNo	oName
CO40	Tina Murphy
CO93	Tony Shaw

5. Boyce-Codd Normal Form (BCNF)

Introduction

BCNF: A relation is in BCNF, if and only if, every **determinant** is a **candidate key.**

- A determinant is an attribute, or a group of attributes, on which some other attribute is functionally dependent.
- To test whether a relation is in BCNF, first identify all the determinants and make sure that they are candidate keys.
- For a functional dependency A→B,
  - 3NF allows this dependency in a relation if **B** is a primary-key attribute and **A** is not a candidate key
  - BCNF insists that for this dependency to remain in a relation, **A** must be a candidate key.

Introduction

- Therefore, BCNF is a stronger form of 3NF, such that every relation in BCNF is also in 3NF.
- However, a relation in 3NF is not necessarily in BCNF.
- Violation of BCNF is quite rare, since it may only happen under specific conditions.
- The potential to violate BCNF may occur when:
  - The relation contains two (or more) composite candidate keys; or
  - The candidate keys overlap, that is have at least one attribute in common.

Previous example

- The Client, PropertyForRent, and Owner relations are all in BCNF, as each relation only has a single determinant, which is the candidate key.
- However, Rental relation contains the three determinants:

(clientNo, propertyNo), (clientNo, rentStart), and (propertyNo, rentStart), as shown below:

fd1 clientNo, propertyNo → rentStart, rentFinish

fd5' clientNo, rentStart → propertyNo, rentFinish

fd6' propertyNo, rentStart → clientNo, rentFinish

- As the three determinants are also candidate keys, the Rental relation is also already in BCNF.

Client

clientNo	cName
CR76	John Kay
CR56	Aline Stewart

Rental

clientNo	propertyNo	rentStart	rentFinish
CR76	PG4	1-Jul-03	31-Aug-04
CR76	PG16	1-Sep-04	1-Sep-05
CR56	PG4	1-Sep-02	10-Jun-03
CR56	PG36	10-Oct-03	1-Dec-04
CR56	PG16	1-Nov-05	10-Aug-06

PropertyForRent

propertyNo	pAddress	rent	ownerNo
PG4	6 Lawrence St, Glasgow	350	CO40
PG16	5 Novar Dr, Glasgow	450	CO93
PG36	2 Manor Rd, Glasgow	375	CO93

Owner

ownerNo	oName
CO40	Tina Murphy
CO93	Tony Shaw

Example - BCNF

Consider the ClientInterview relation that shows the details of client interviews conducted by members of staff. The members of staff involved in interviewing clients are allocated to a specific room on the day of the interview. However, a room may be allocated to several members of staff as required throughout a working day. A client is only interviewed once on a given date, but may be requested to attend further interviews at later dates.

The ClientInterview relation has the following form:

ClientInterview (clientNo, interviewDate, interviewTime, staffNo, roomNo)

The functional dependencies in ClientInterview:

fd1	clientNo, interviewDate → interviewTime, staffNo, roomNo	(Primary key)
fd2	staffNo, interviewDate, interviewTime → clientNo	(Candidate key)
fd3	roomNo, interviewDate, interviewTime → staffNo, clientNo	(Candidate key)
fd4	staffNo, interviewDate → roomNo	

ClientInterview

clientNo	interviewDate	interviewTime	staffNo	roomNo
CR76	13-May-05	10.30	SG5	G101
CR56	13-May-05	12.00	SG5	G101
CR74	13-May-05	12.00	SG37	G102
CR56	1-Jul-05	10.30	SG5	G102

Example - BCNF

fd1	clientNo, interviewDate → interviewTime, staffNo, roomNo	(Primary key)
fd2	staffNo, interviewDate, interviewTime → clientNo	(Candidate key)
fd3	roomNo, interviewDate, interviewTime → staffNo, clientNo	(Candidate key)
fd4	staffNo, interviewDate → roomNo	

- The primary key: (clientNo, interviewDate)
- Candidate keys:

  (clientNo, interviewDate)

  (staffNo, interviewDate, interviewTime)

  (roomNo, interviewDate, interviewTime)
- Thus, the ClientInterview relation has **three** composite candidate keys, which overlap by sharing the common attribute ***interviewDate***.
- As functional dependencies fd1, fd2, and fd3 are all candidate keys for this relation, none of these dependencies will cause problems for the relation.
- The only functional dependency that requires discussion is,

  (staffNo, interviewDate) → roomNo (fd4)

Example - BCNF

- Even though (staffNo, interviewDate) is not a candidate key for the ClientInterview relation this functional dependency is allowed in 3NF because roomNo is a primary-key attribute being part of the candidate key (roomNo, interviewDate, interviewTime).

Candidate keys:

(clientNo, interviewDate)
(staffNo, interviewDate, interviewTime)
(roomNo,interviewDate, interviewTime)

fd1	clientNo, interviewDate → interviewTime, staffNo, roomNo	(Primary key)
fd2	staffNo, interviewDate, interviewTime → clientNo	(Candidate key)
fd3	roomNo, interviewDate, interviewTime → staffNo, clientNo	(Candidate key)
fd4	staffNo, interviewDate → roomNo	

- As there are no partial or transitive dependencies on the primary key (clientNo, interviewDate), and functional dependency fd4 is allowed, the ClientInterview relation is in 3NF.
- However, this relation is not in BCNF due to the presence of the (staffNo, interviewDate) determinant, which is not a candidate key for the relation.

Example - BCNF

- BCNF requires that all determinants in a relation must be a candidate key for the relation.
- As a consequence the ClientInterview relation may suffer from update anomalies.

Example: To change the room number for staff number SG5 on the 13-May-05 you must update two tuples. If only one tuple is updated with the new room number, this results in an inconsistent state for the database.

ClientInterview

clientNo	interviewDate	interviewTime	staffNo	roomNo
CR76	13-May-05	10.30	SG5	G101
CR56	13-May-05	12.00	SG5	G101
CR74	13-May-05	12.00	SG37	G102
CR56	1-Jul-05	10.30	SG5	G102

Example - BCNF

- To transform the ClientInterview relation to BCNF, you must remove the violating functional dependency by creating two new relations called Interview and StaffRoom.

  Interview (clientNo, interviewDate, interviewTime, staffNo)

  StaffRoom (staffNo, interviewDate, roomNo)

Interview

clientNo	interviewDate	interviewTime	staffNo
CR76	13-May-05	10.30	SG5
CR56	13-May-05	12.00	SG5
CR74	13-May-05	12.00	SG37
CR56	1-Jul-05	10.30	SG5

StaffRoom

staffNo	interviewDate	roomNo
SG5	13-May-05	G101
SG37	13-May-05	G102
SG5	1-Jul-05	G102

ClientInterview

clientNo	interviewDate	interviewTime	staffNo	roomNo
CR76	13-May-05	10.30	SG5	G101
CR56	13-May-05	12.00	SG5	G101
CR74	13-May-05	12.00	SG37	G102
CR56	1-Jul-05	10.30	SG5	G102

Example - BCNF

- However, it may not always be desirable to transform a relation into BCNF.

  Example: if there is a functional dependency that is not preserved when we perform the decomposition (when the determinant and the attributes it determines are placed in different relations).

- When this occurs, it may be better to stop at 3NF, which always preserves dependencies.

  Example: In creating the two BCNF relations from the original ClientInterview relation, we have 'lost' the functional dependency, roomNo, interviewDate, interviewTime → staffNo, clientNo (fd3), as the determinant for this dependency is no longer in the same relation. However, we must recognize that if the functional dependency, staffNo, interviewDate→roomNo (fd4) is not removed, the ClientInterview relation will have data redundancy.

Pros & Cons in Normalization

Advantages

- Reduction of data redundancy within tables
- Reduce data storage space
- Reduce inconsistency of data
- Reduce update cost
- Remove many-to-many relationship
- Improve the flexibility of the system

Disadvantages

- Increase join
- Reduce efficiency
- Increase use of indexes
- Increase complexity of the system

Summary

- **UNF** is a table that contains one or more repeating groups.
- **1NF** is a relation in which the intersection of each row and column contains one and only one value.
- **2NF** is a relation that is in 1NF and every non-primary-key attribute is fully functionally dependent on the *primary key*.

  Full functional dependency indicates that if A and B are attributes of a relation, B is fully functionally dependent on A if B is functionally dependent on A but not on any proper subset of A.

- **3NF** is a relation that is in 1NF and 2NF in which no non-primary key attribute is transitively dependent on the *primary key*.

  Transitive dependency is a condition where A, B, and C are attributes of a relation such that if A → B and B → C, then C is transitively dependent on A via B (provided that A is not functionally dependent on B or C).

- **BCNF** is a relation in which every determinant is a candidate key.

Any Questions?